

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине: «КиСБ»

Тема: «Исследование алгоритмов генерации псевдослучайных последовательностей и методов тестирования чисел на простоту»

Выполнил:
Студент 4-го курса
Группы АС-65
Гуща И.В.
Проверил:
Хацкевич А.С.

Брест 2026

Цель работы: освоить программную реализацию и практическое применение алгоритмов генерации псевдослучайных последовательностей (РСП), методов их статистической оценки, а также вероятностных и детерминированных тестов простоты чисел с построением доказуемо простых чисел.

Вариант 3.

1. Генерация и тестирование псевдослучайных последовательностей (файл "Лабораторная работа №1 (ч.1)")

В рамках данного этапа необходимо разработать программный модуль генерации последовательностей и провести их первичный анализ.

```
a=9
c=14
N=256
x0=1
def obr(x0):
    return pow(x0,-1,N)
def gen(x0):
    if x0:
        x1=(a*obr(x0)+c)%N
    else:
        x1=c
    return x1
if __name__ == '__main__':
    generated=[]
    for i in range(0,1000):
        x0=gen(x0)
        generated.append(x0)

    print(generated)

    M=sum(generated) / len(generated)
    print(f"Мат.Ожидание {sum(generated) / len(generated)}")
    s=0
    for g in generated:
        s+=(M-g)**2
    s/=len(generated)
    print(f"Дисперсия {s}")
    print(f"Для равномерного Мат.Ожидание =
    {(max(generated)+min(generated))/2} Дисперсия = {(max(generated)-
    min(generated))**2/12}")
```

Мат.Ожидание 127.9968

Дисперсия 5464.385589760022

Для равномерного Мат.Ожидание = 128.0 Дисперсия = 5376.333333333333

```
a=17
c=16
N=256
x0=11
```

[227, 139, 99, 11, 227, 139, 99, 11, 227, 139, 99, 11, 227, 139, 99, 11, 227...]

Последовательность имеет достаточно малый период

Мат.Ожидание 119.0

Дисперсия 6032.0

Для равномерного Мат.Ожидание = 119.0 Дисперсия = 3888.0

2. Тестирование на простоту и генерация больших простых чисел (файл "Лабораторная работа №1 (ч.2)")

На втором этапе сгенерированные на первом этапе псевдослучайные числа (или базовые механизмы генерации) интегрируются в модуль поиска и проверки чисел на простоту.

Полиномиальная проверка на простоту

```

def isProstoe(n):
    if n < 2:
        return False
    if not prover_del(n):
        return False

    if isPerfectPower(n):
        return False

    r = findR(n)
    print("n =", n, "r =", r)
    for a in range(2, r + 1):
        if NOD(a, n) > 1:
            return False

    if n <= r:
        return True

    limit = math.floor(
        math.sqrt(phi(r)) * math.log2(n)
    )

    # Проверяем:
    for a in range(1, limit + 1):
        polynomial = [0] * r
        polynomial[0] = a % n
        polynomial[1] = 1

        left = polynomial_power(
            polynomial,
            n,
            r,
            n
        )

        right = [0] * r

        right[0] = a % n
        right[n % r] = (right[n % r] + 1) % n

        if left != right:
            return False
    return True
if __name__ == '__main__':
    x=x0
    while True:
        x, n=candidate(17,x)
        #print(n)
        if(isProstoe(n)):
            print(n)
            break

```

n = 129023 r = 293

129023 – Простое

Вывод : при генерации простых чисел псевдослучайным методом, необходимо проверить число на простоту. Для этого в лабораторной работе использовался полиномиальный алгоритм